Algorithms and Complexity

Intro to Algorithms

An ***algorithm*** is a sequence of steps for accomplishing a task.

E.g. Search Algorithms

Linear search is a search algorithm that starts from the beginning of a list, and checks each element until the search key is found or the end of the list is reached.

Binary search is a faster algorithm for searching a list if the list's elements are sorted and directly accessible (such as an array).

It first checks the middle element of the list. If the search key is found, the algorithm returns the matching location. If the search key is not found, the algorithm repeats the search on the remaining left sublist (if the search key was less than the middle element) or the remaining right sublist (if the search key was greater than the middle element).

Complexity

- As you progress in this class, you're writing increasingly complex programs. You're also seeing that there are often many ways to solve the same problem. *How do you compare different solutions to the same problem to see which is better?*
- We desire algorithms that solve problems quickly or with high efficiency.
- ***Complexity***
  - A measure of the computing resources that are used by a piece of code, such as time, memory, or disk space.
- An algorithm with higher complexity uses more time or resources to solve a problem.

Time and Space Complexities

Sometimes, there are more than one way to solve a problem. We need to learn how to compare the performance different algorithms and choose the best one to solve a particular problem. While analyzing an algorithm, we mostly consider time complexity and space complexity.

Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input.

Space complexity of an algorithm quantifies the amount of space or memory taken by an algorithm to run as a function of the length of the input.

Empirical Analysis

- One way to determine an algorithm's approximate time complexity is to program it, run the program, and measure how long it takes to run. This is sometimes called an ***empirical analysis*** of the algorithm.
- E.g.
  - You could empirically analyze *linear search* and *binary search* by writing both as programs, running them on the same input, and timing them.

- *Problem: not very reliable. On a different computer with a different processor speed and different memory the program might not take the same amount of time.*

Algorithm Analysis

A more neutral way to measure a program's performance is to examine its code or pseudocode and roughly count the number of statements that are executed.

Algorithm Analysis

> the practice of applying techniques to mathematically approximate the performance of various computing algorithms.

Constant time operations

- For the purpose of simplification, let's assume that the following actions require an *equal and fixed amount of time to execute*:
  - Variable declarations and assignments
  - Evaluating mathematical and logical expressions
  - Accessing or modifying an individual element of an array
  - Simple method calls (where the method does not perform a loop)
- From the preceding simple rules, we can extrapolate the runtimes of larger and more complex pieces of code.
  - For example, the runtime of a group of statements in sequential order is the sum of the individual runtimes of the statements:

```
Statement1;
Statement2;
Statement3;
```

3

Constant time operations

Operation	Example

Addition, subtraction, multiplication, and division of fixed size integer or floating point values.	`w = 10.4`
	`x = 3.4`
	`y = 2.0`
	`z = (w - x) / y`

Assignment of a reference, pointer, or other fixed size data value.	`x = 1000`
	`y = x`
	`a = true`
	`b = a`

Comparison of two fixed size data values.	`a = 100`
	`b = 200`
	`if (b > a) {`
	`  ...`
	`}`

Read or write an array element at a particular index.	`x = arr[index]` `arr[index + 1] = x + 1`

Growth functions

- The runtime of a loop is roughly equal to the runtime of its body times the number of iterations of the loop.
- For example, a loop with a body that contains *K* simple statements and that repeats *N* times will have a runtime of roughly *(K * N)*:

```
for(N times) {
    statement1;
    statement2;
    statement3;
}
```

$3N$, since $K = 3$

Growth functions

The runtime of multiple loops placed sequentially (not nested) with other statements is the sum of the loops' runtimes and the other statements' runtimes:

```
statement1;
for (N times) {
    statement2;
}
for (M times) {
    statement3;
}
statement4;
statement5;
```

N

M

$N + M + 3$

Lower and Upper bound

Lower bound is the minimum time required to run an algorithm.

Upper bound is the maximum time required to run an algorithm.

```
public static int unknown (int arr[], int x)
{
    int n = arr.length;
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == x)
            return i;
    }
    return -1;
}
```

O notation provides a growth rate for an algorithm's upper bound.
Ω notation provides a growth rate for an algorithm's lower bound.
Θ notation provides a growth rate that is both an upper and lower bound.

Big O notation

- Normally, the loops in long-running algorithms are processing some kind of data. Many algorithms run very quickly if the input dataset is small, so we generally worry about the performance only for large datasets.
- E.g. consider the following set of loops that process an array of N elements

```
Statement0;
for(N times){
    for(N times){
        statement1;
    }
}
for(N times){
    statement2;
    statement3;
}
Statement4;
```

N^2

N

$N^2 + N$

Complexity classes

- Here are some of the most common complexity classes, listed in order from slowest to fastest growth (i.e., *from lowest to highest complexity*):

Constant-time, or $O(1)$: algorithms have runtimes that don't depend on input size.

- E.g. if a user input is greater than a certain number

```
FindMin(x, y) {
   if (x < y) {
      return x
   }
   else {
      return y
   }
}
```

Complexity classes

Linear, or $O(N)$***:*** algorithms have runtimes that are *directly proportional to N* (i.e., roughly they double when *N* doubles). Many algorithms that process each element of a list/array are linear.

E.g. Count the appearance of a certain element in a list, sum of a list, average of a list, find max, find min, linear search, etc.

```
LinearSearch(numbers, numbersSize, key) {
  for (i = 0; i < numbersSize; ++i) {
      if (numbers[i] == key) {
         return i
      }
   }

   return -1 // not found
}
```

Complexity classes

Logarithmic, or $O(Log\ N)$***:*** algorithms typically divide a problem space in half repeatedly until the problem is solved.

- E.g. Binary Search

```
BinarySearch(numbers, N, key) {
   mid = 0
   low = 0
   high = N - 1

   while (high >= low) {
      mid = (high + low) / 2
      if (numbers[mid] < key) {
         low = mid + 1
      }
      else if (numbers[mid] > key) {
         high = mid - 1
      }
      else {
         return mid
      }
   }

   return -1   // not found
}
```

Complexity classes

Log-linear, or $O(NLog\ N)$: algorithms typically perform a combination of logarithmic and linear operations, such as executing a logarithmic algorithm over every element of a dataset of size N.

E.g. Many efficient sorting algorithms are Log-linear (Merge Sort)

```
MergeSort(numbers, i, k) {
   j = 0
   if (i < k) {
      j = (i + k) / 2              // Find midpoint

      MergeSort(numbers, i, j)     // Sort left part
      MergeSort(numbers, j + 1, k) // Sort right part
      Merge(numbers, i, j, k)      // Merge parts
   }
}
```

Complexity classes

Quadratic, or $O(N^2)$: algorithms have runtimes that are proportional to the square of the input size. This means that quadratic algorithms'runtimes roughly quadruple when N doubles.

E.g. A solution for twoSum(), selection sort

```
SelectionSort(numbers, numbersSize) {
   for (i = 0; i < numbersSize; ++i) {
      indexSmallest = i
      for (j = i + 1; j < numbersSize; ++j) {
         if (numbers[j] < numbers[indexSmallest]) {
            indexSmallest = j
         }
      }

      temp = numbers[i]
      numbers[i] = numbers[indexSmallest]
      numbers[indexSmallest] = temp
   }
}
```

Complexity classes

Exponential, or $O(2^N)$**:** algorithms have runtimes that are proportional to 2 raised to the power of the input size. This means that if the input size increases by just one,the algorithm will take roughly twice as long to execute.

- E.g. print the "power set" of a dataset, which is the set of all possible subsets of the data.

```
Fibonacci(N) {
  if ((1 == N) || (2 == N)) {
     return 1
  }
  return Fibonacci(N-1) + Fibonacci(N-2)
}
```